

React Lab: Forms, Controlled Components, and Input Controls

Hands-on Lab Solution — "ticketraisingapp" (ComplaintRegister Component)

Objectives

- Explain about React forms
- Define controlled components
- Explain about various input controls
- Explain about handling forms
- Explain about submitting forms

Lab Goals

- Implement React forms
- Use various input controls like textbox, button, and textarea

Prerequisites

- Node.js
- NPM
- Visual Studio Code

1. React Forms

A React form works much like an HTML form — it uses the same elements such as `<input>`, `<textarea>`, `<select>`, and `<button>` — but React usually keeps the form's data in the component's own state rather than letting the DOM manage it. This gives the component full, real-time access to whatever the user has typed, so it can validate, transform, or react to input as it happens, instead of only reading the values once the form is submitted.

2. Controlled Components

A controlled component is a form element (input, textarea, select, etc.) whose displayed value is driven entirely by React state, rather than by the element's own internal DOM state. The element's value prop is always set from `this.state`, and every keystroke fires an `onChange` handler that updates that state with `setState()` — which then re-renders the element with the new value. React's state becomes the single source of truth for what's on screen.

```
<input
  type="text"
  value={this.state.employeeName}      // value comes FROM state
  onChange={this.handleNameChange}    // every change updates state
/>
```

This is different from an uncontrolled component, where the DOM itself keeps track of the value and React only reads it on demand (typically via a ref). Controlled components are the recommended default in React because they make validation, conditional disabling, and formatting straightforward.

3. Various Input Controls

React supports the same set of form controls as HTML, each used as a controlled component in the same pattern (value + onChange):

Control	Typical Use
<code><input type="text"></code>	Single-line text entry, e.g. the employee name textbox in this lab.
<code><textarea></code>	Multi-line text entry, e.g. the complaint description in this lab.
<code><button></code>	Triggers an action, e.g. submitting the form or resetting it.
<code><input type="checkbox"></code>	Boolean on/off choice; controlled via the checked prop instead of value.
<code><input type="radio"></code>	Single choice among a group of mutually exclusive options.
<code><select></code>	Dropdown choice among a predefined list of options.

4. Handling Forms

Handling a form means wiring up onChange handlers for each controlled input so the component's state always mirrors what the user has typed. Each field typically gets its own handler (or a single generic handler keyed off event.target.name for larger forms) that calls setState() with the new value read from event.target.value.

```
handleNameChange(event) {
  this.setState({ employeeName: event.target.value });
}

handleComplaintChange(event) {
  this.setState({ complaint: event.target.value });
}
```

5. Submitting Forms

A form is submitted by attaching an onSubmit handler to the <form> element itself (rather than only an onClick on the button), so the Enter key also triggers submission the way users expect. Because a native HTML form submission normally causes a full page reload, the very first thing the handler does is call event.preventDefault(). After that, the handler can validate the state, process the data (in this lab, generating a reference number), and give the user feedback.

```
handleSubmit(event) {
  event.preventDefault(); // stop the default full-page form submission
  // ... validate and process this.state here ...
}
```

```
<form onSubmit={this.handleSubmit}>
  {/* fields */}
  <button type="submit">Submit Complaint</button>
</form>
```

Lab Steps

Step 1: Create the React application

```
npx create-react-app ticketraisingapp
cd ticketraisingapp
npm start
```

Open the ticketraisingapp folder in Visual Studio Code. Inside the src folder, create a new file: ComplaintRegister.js.

Step 2: Create the ComplaintRegister component

ComplaintRegister keeps employeeName and complaint in state as controlled inputs. On submit, handleSubmit prevents the page reload, validates that both fields are filled in, generates a random reference number, and shows it to the user in an alert box before clearing the form.

src/ComplaintRegister.js

```
import React, { Component } from 'react';

class ComplaintRegister extends Component {
  constructor(props) {
    super(props);
    this.state = {
      employeeName: '',
      complaint: ''
    };

    this.handleChange = this.handleChange.bind(this);
    this.handleComplaintChange = this.handleComplaintChange.bind(this);
    this.handleSubmit = this.handleSubmit.bind(this);
  }

  // keeps employeeName in sync with the textbox (controlled component)
  handleChange(event) {
    this.setState({ employeeName: event.target.value });
  }

  // keeps complaint in sync with the textarea (controlled component)
  handleComplaintChange(event) {
    this.setState({ complaint: event.target.value });
  }

  // creates a simple reference number, e.g. REF4821
  generateReferenceNumber() {
    const randomNum = Math.floor(1000 + Math.random() * 9000);
    return 'REF' + randomNum;
  }

  handleSubmit(event) {
```

```

    event.preventDefault(); // prevent the default full-page form submission

    if (!this.state.employeeName.trim() || !this.state.complaint.trim()) {
        alert('Please fill in both Employee Name and Complaint.');
```

```

        return;
    }

    const refNumber = this.generateReferenceNumber();
    alert(
        'Complaint submitted successfully!\n' +
        'Employee: ' + this.state.employeeName + '\n' +
        'Reference Number: ' + refNumber
    );

    // reset the form after submission
    this.setState({ employeeName: '', complaint: '' });
}

render() {
    return (
        <div style={{ padding: '20px', fontFamily: 'Arial, sans-serif' }}>
            <h2>Complaint Register</h2>
            <form onSubmit={this.handleSubmit}>
                <div style={{ marginBottom: '10px' }}>
                    <label>Employee Name: </label>
                    <input
                        type="text"
                        value={this.state.employeeName}
                        onChange={this.handleNameChange}
                        placeholder="Enter employee name"
                    />
                </div>

                <div style={{ marginBottom: '10px' }}>
                    <label>Complaint: </label>
                    <br />
                    <textarea
                        value={this.state.complaint}
                        onChange={this.handleComplaintChange}
                        placeholder="Describe your complaint"
                        rows="5"
                        cols="40"
                    />
                </div>

                <button type="submit">Submit Complaint</button>
            </form>
        </div>
    );
}
}

export default ComplaintRegister;

```

Step 3: Update App.js

src/App.js

```
import React from 'react';
import ComplaintRegister from './ComplaintRegister';

function App() {
  return (
    <div className="App">
      <ComplaintRegister />
    </div>
  );
}

export default App;
```

Step 4: Render with ReactDOM

src/index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Step 5: Run the application

```
npm start
```

This opens the app at <http://localhost:3000>.

Expected Output

```
Complaint Register

Employee Name: [ John Mathew ]

Complaint:
[ My laptop keeps restarting during video calls. ]
[ ]

[ Submit Complaint ]
```

After filling in both fields and clicking Submit Complaint, an alert box appears showing a message similar to:

```
Complaint submitted successfully!
Employee: John Mathew
Reference Number: REF4821
```

If either field is left empty, clicking Submit Complaint instead shows an alert asking the user to fill in both fields, and the form is not cleared.

Summary

- React forms reuse standard HTML elements, but the component's state — not the DOM — becomes the source of truth for their values.

- Controlled components tie an input's value prop to state and update that state on every onChange, keeping the UI and the data perfectly in sync.
- This lab used a text input (employee name) and a textarea (complaint description) as controlled input controls, plus a submit button.
- Handling a form means wiring an onChange handler per field so state is updated as the user types.
- Submitting a form means attaching onSubmit to the <form> element, calling event.preventDefault() to stop the page reload, then validating and processing the data — here, generating and displaying a reference number in an alert box.